

STM32G474RE

FreeRTOS 적용 코드 해설서

CubeMX 설정부터 스케줄러, 태스크, 큐, 이벤트, Mutex,
SysTick/PendSV/SVC 문맥 전환까지 현재 코드로 이해하기



이 문서의 목표

FreeRTOS 기능 목록을 외우는 것이 아니라, 현재 프로젝트의 어떤 코드가 어떤 커널 개념을 구현하는지 연결해서 이해하는 것입니다.

대상 보드: STM32 NUCLEO-G474RE
RTOS: FreeRTOS Kernel V10.3.1 / CMSIS-RTOS V2
기준 소스: C:\project\NUCLEOG474RE

문서 구성

문서 구성	2
1. CubeMX에서 추가한 FreeRTOS 설정	3
2. 현재 코드에서 커널을 시작하는 과정	4
3. 태스크를 만드는 코드와 정적 메모리	5
4. 스케줄러가 태스크를 번갈아 실행하는 방법	7
5. EventFlags: 버튼과 센서를 태스크에 전달	8
6. Message Queue와 ACK Queue: 모터 명령 전달	9
7. Mutex: 여러 태스크가 공유하는 드라이버 보호	11
8. 인터럽트와 태스크를 연결하는 코드	12
9. SysTick, PendSV, SVC가 만드는 문맥 전환	13
10. 커널이 자동 생성하는 내부 태스크	14
11. 내가 직접 추가한 코드 지도	15
12. 현재 코드에서 기억할 주의점	16

먼저 알아둘 구분

CubeMX가 생성한 `src/lib/cube_g474/Core`의 `defaultTask`는 현재 실제 빌드에서 사용되지 않습니다. 현재 실행되는 태스크는 `src/main.c`, `src/bsp/rtos.c`, `src/ap/task`에 직접 작성한 코드입니다.

1. CubeMX에서 추가한 FreeRTOS 설정

CubeMX의 .ioc 파일에는 FreeRTOS 활성화, CMSIS V2 인터페이스, 기본 태스크, heap 크기, HAL timebase 변경 내용이 남아 있습니다.

cube_g474.ioc - FreeRTOS 관련 핵심 설정

```
FREERTOS.Tasks01=defaultTask,24,128,StartDefaultTask,...,Dynamic,...
FREERTOS.configCHECK_FOR_STACK_OVERFLOW=2
FREERTOS.configTOTAL_HEAP_SIZE=48360
FREERTOS.configUSE_MALLOC_FAILED_HOOK=1
VP_FREERTOS_VS_CMSIS_V2.Mode=CMSIS_V2

NVIC.TimeBase=TIM7_DAC_IRQn
NVIC.TimeBaseIP=TIM7
```

소스: stm32g474/src/lib/cube_g474/cube_g474.ioc:24-30, 82-93, 260

CubeMX GUI에서 의미하는 선택

CubeMX 항목	설정값	코드에 미친 영향
Middleware / FreeRTOS	Enabled	커널, CMSIS wrapper, 설정 파일 생성
Interface	CMSIS V2	osThreadNew, osDelay 같은 CMSIS API 사용
defaultTask	Normal / 128 word / Dynamic	CubeMX 원본 app_freertos.c에 기본 태스크 생성
Total Heap	48,360 byte	동적 생성 객체가 사용하는 heap_4 영역
Stack overflow	Mode 2	태스크 전환 시 스택 경계 검사
SYS Timebase	TIM7	SysTick을 RTOS 전용으로 분리

CubeMX가 자동 생성한 스케줄러 코드

CubeMX Core/Src/main.c - 자동 생성 원본

```
osKernelInitialize();
MX_FREERTOS_Init();
osKernelStart();

while (1)
{
}
```

소스: stm32g474/src/lib/cube_g474/Core/Src/main.c:103-119

CubeMX Core/Src/app_freertos.c - defaultTask 생성

```
const osThreadAttr_t defaultTask_attributes = {
    .name = "defaultTask",
    .priority = (osPriority_t)osPriorityNormal,
    .stack_size = 128 * 4
};

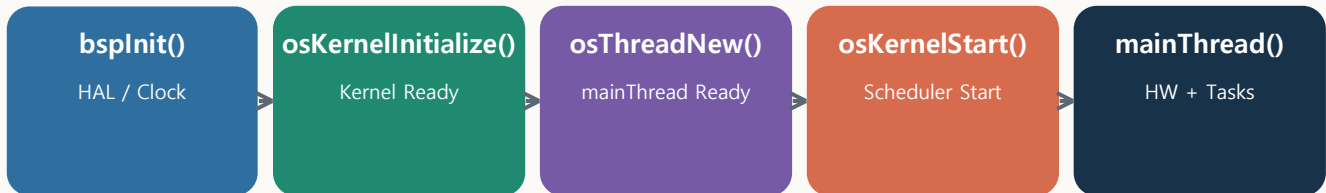
defaultTaskHandle =
    osThreadNew(StartDefaultTask, NULL, &defaultTask_attributes);
```

소스: stm32g474/src/lib/cube_g474/Core/Src/app_freertos.c:50-56, 101-128

현재 프로젝트에서는 이 defaultTask를 사용하지 않음

빌드 설정에서 src/lib/cube_g474/Core가 제외되어 있고, 현재 main.c는 MX_FREERTOS_Init()을 호출하지 않습니다. CubeMX 코드는 참고 원본이고 실제 실행 구조는 직접 작성한 코드입니다.

2. 현재 코드에서 커널을 시작하는 과정



src/main.c - 실제 실행되는 시작 코드

```

int main(void)
{
    bspInit();
    osKernelInitialize();

    if (osThreadNew(mainThread, NULL, rtosGetMainThreadAttr()) == NULL)
    {
        ledInit();
        while (1)
        {
            ledToggle(_DEF_LED1);
            delay(50);
        }
    }

    osKernelStart();

    while (1)
    {
    }
}
  
```

소스: stm32g474/src/main.c:13-34

각 호출이 커널에서 의미하는 것

코드	직후 커널 상태	실제 의미
bspInit()	RTOS 시작 전	HAL, Clock, GPIO clock과 TIM7 HAL tick 준비
osKernelInitialize()	Ready	CMSIS 커널 상태를 Inactive에서 Ready로 변경
osThreadNew()	태스크 Ready	TCB와 스택을 등록하지만 아직 실행하지 않음
osKernelStart()	Running	vTaskStartScheduler를 호출하고 CPU 제어권을 스케줄러에 넘김

cmsis_os2.c - osKernelStart 내부

```
if (KernelState == osKernelReady)
{
    SVC_Setup();
    KernelState = osKernelRunning;
    vTaskStartScheduler();
}
```

소스: stm32g474/src/bsp/FreeRTOS/Source/CMSIS_RTOS_V2/cmsis_os2.c:265-285

osKernelStart 아래 while(1)이 비어 있는 이유

정상적으로 스케줄러가 시작되면 osKernelStart()는 돌아오지 않습니다. 그 아래 무한 루프는 커널 시작 실패 등 비정상 상황을 위한 안전 장치입니다.

3. 태스크를 만드는 코드와 정적 메모리

현재 프로젝트의 애플리케이션 태스크는 TCB와 스택을 전역 정적 메모리로 미리 확보합니다. 이 방식은 실행 중 heap 부족으로 태스크 생성이 실패할 가능성을 줄입니다.

rtos.c - Button 태스크의 TCB, 스택, 속성

```
static StaticTask_t threadButton_cb;

static StackType_t threadButton_stack[
    _HW_DEF_RTOS_THREAD_MEM_BUTTON / sizeof(StackType_t)
];

static const osThreadAttr_t threadButton_attributes =
{
    .name      = "threadButton",
    .cb_mem    = &threadButton_cb,
    .cb_size   = sizeof(threadButton_cb),
    .stack_mem = threadButton_stack,
    .stack_size = sizeof(threadButton_stack),
    .priority  = _HW_DEF_RTOS_THREAD_PRI_BUTTON,
};
```

소스: stm32g474/src/bsp/rtos.c:81-92

필드	의미	커널 사용처
cb_mem	Task Control Block 메모리	상태, 우선순위, 현재 스택 포인터 등을 저장
stack_mem	태스크 전용 스택	지역변수, 함수 복귀 주소, 저장된 CPU 레지스터
stack_size	스택 크기(byte)	CMSIS wrapper가 ARM StackType_t 단위로 변환
priority	스케줄링 우선순위	Ready 태스크 중 실행 대상을 선택

실제로 태스크를 커널에 등록하는 한 줄

task_button.c - 태스크 생성

```
bool taskButtonInit(void)
{
    return osThreadNew(
        threadButton,
        NULL,
        rtosGetButtonThreadAttr()
    ) != NULL;
}
```

소스: stm32g474/src/ap/task/task_button.c:24-27

CMSIS wrapper - 정적 메모리가 있으면

```
hTask = xTaskCreateStatic(
    (TaskFunction_t)func,
    name,
    stack,
    argument,
    prio,
    (StackType_t *)attr->stack_mem,
    (StaticTask_t *)attr->cb_mem
);
```

소스: stm32g474/src/bsp/FreeRTOS/Source/CMSIS_RTOS_V2/cmsis_os2.c:472-505

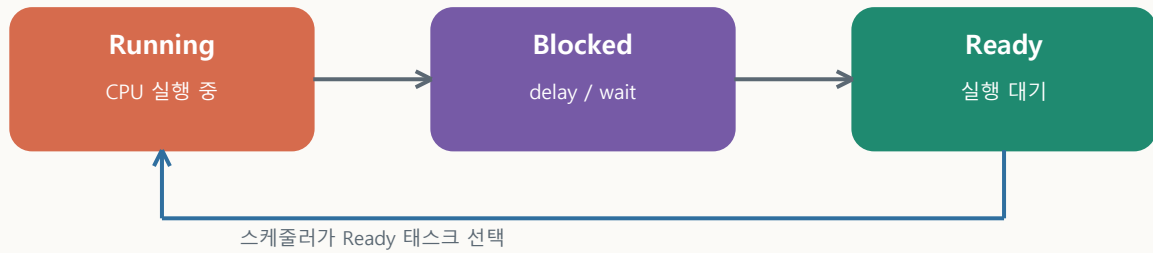
현재 생성되는 애플리케이션 태스크

태스크	우선순위	스택	주요 대기 방식	역할
mainThread	Normal	2048 B	delay(1)	HW/AP 초기화와 CLI
threadLed	Low	512 B	delay(500)	상태 LED 점멸
threadButton	Normal	512 B	osDelay(10)	버튼 디바운싱과 이벤트 발생
threadSensor	Normal	512 B	osDelay(10)	센서 상태 동기화
threadStepMotor	Normal	1024 B	Event/1ms delay	비동기 chunk 모터 제어
threadSequence	Normal	1024 B	Event wait forever	장비 상태 머신

이름에 task가 있어도 모두 스레드는 아님

task_servo.c, task_valve.c, task_pump.c, task_dcmotor.c는 별도 osThreadNew()가 없습니다. 현재는 드라이버 호출을 감싸는 API 계층입니다.

4. 스케줄러가 태스크를 번갈아 실행하는 방법



CPU는 한 번에 하나의 태스크만 실행합니다. FreeRTOS는 각 태스크의 상태를 관리하고 Ready 상태 태스크 중 우선순위가 가장 높은 태스크를 선택합니다.

상태	의미	현재 코드에서 만드는 동작
Running	현재 CPU를 사용하는 태스크	버튼 읽기, 센서 확인, 큐 처리
Ready	실행 가능하지만 CPU를 기다림	delay가 끝났거나 이벤트가 도착한 태스크
Blocked	시간 또는 신호를 기다림	osDelay, osEventFlagsWait, QueueGet(timeout)
Suspended	명시적으로 중지됨	현재 애플리케이션에서는 직접 사용하지 않음

osDelay는 CPU를 버리는 코드가 아니라 반납하는 코드

task_button.c - 10ms 주기

```

while (1)
{
    // RESET / STOP / START / FOOT input scan
    ...
    osDelay(BUTTON_SCAN_MS);    // BUTTON_SCAN_MS = 10
}
  
```

cmsis_os2.c - osDelay 내부

```

osStatus_t osDelay(uint32_t ticks)
{
    if (ticks != 0U)
    {
        vTaskDelay(ticks);
    }
    return osOK;
}
  
```

소스: stm32g474/src/bsp/FreeRTOS/Source/CMSIS_RTOS_V2/cmsis_os2.c:881-896

현재 태스크 실행 예

순서 예	실행 코드	결과
1	Button: 입력 확인 후 osDelay(10)	Button이 Blocked로 이동
2	Sensor: 상태 확인 후 osDelay(10)	Sensor가 Blocked로 이동
3	Sequence: appEventWait(..., osWaitForever)	이벤트까지 Blocked
4	StepMotor: 할 일 없으면 osDelay(1)	1ms 동안 Blocked
5	mainThread: CLI 후 delay(1)	1ms 동안 Blocked
6	Ready 태스크가 없을 때	커널 Idle Task 실행

5. EventFlags: 버튼과 센서를 태스크에 전달

EventFlags는 하나의 비트 집합을 여러 태스크와 ISR이 공유하는 신호판입니다. 각 bit가 RESET, STOP, 센서 감지와 같은 하나의 사건을 나타냅니다.

rtos.h - 이벤트 비트 정의

```
#define APP_EVT_RESET_REQ      (1 << 0)
#define APP_EVT_STOP_REQ      (1 << 1)
#define APP_EVT_START_REQ     (1 << 2)
#define APP_EVT_FOOT_PRESS    (1 << 3)
#define APP_EVT_SN04_1_DETECTED (1 << 4)
#define APP_EVT_SN04_2_DETECTED (1 << 5)
#define APP_EVT_STEP_MOTOR_DONE (1 << 6)
```

소스: stm32g474/src/bsp/rtos.h:14-21

이벤트 객체 생성

app_event.c

```
static osEventFlagsId_t app_evt = NULL;

bool appEventInit(void)
{
    app_evt = osEventFlagsNew(rtosGetAppEventAttr());
    return app_evt != NULL;
}
```

소스: stm32g474/src/ap/task/app_event.c:10-21

생산자: Button 태스크가 bit를 Set

task_button.c

```
if (buttonUpdate(&start_btn, buttonGetPressed(_DEF_BUTTON3)))
{
    appEventSet(APP_EVT_START_REQ);
}
```


소비자: Sequence 태스크가 bit를 기다림

app_sequence.c

```

evt = appEventWait(
    CONTROL_EVT,
    osFlagsWaitAny | osFlagsNoClear,
    osWaitForever
);

if ((evt & APP_EVT_START_REQ) != 0U)
{
    appEventClear(APP_EVT_START_REQ | APP_EVT_FOOT_PRESS);
    runStartSequence();
}

```

소스: stm32g474/src/ap/app_sequence.c:73-120

옵션	의미	현재 동작
osFlagsWaitAny	지정 bit 중 하나만 발생해도 반환	RESET/STOP/START/FOOT 중 하나 처리
osFlagsNoClear	반환하면서 자동 삭제하지 않음	처리 코드에서 appEventClear로 명시적 삭제
osWaitForever	시간 제한 없이 대기	이벤트가 없을 때 Sequence 태스크 CPU 사용량 0

6. Message Queue와 ACK Queue: 모터 명령 전달

Sequence 태스크가 스텝모터 드라이버를 직접 긴 시간 실행하지 않고, 명령을 큐로 전달합니다. StepMotor 태스크는 명령을 수행한 뒤 결과를 ACK 큐로 돌려줍니다.

rtos.h - 큐에 복사되는 메시지 구조체

```

typedef struct
{
    uint32_t cmd_id;
    rtos_step_motor_cmd_t cmd;
    uint8_t ch;
    int32_t step;
    uint32_t pulse_delay_us;
} rtos_step_motor_msg_t;

typedef struct
{
    uint32_t cmd_id;
    rtos_step_motor_cmd_t cmd;
    rtos_step_motor_ack_result_t result;
} rtos_step_motor_ack_t;

```

소스: stm32g474/src/bsp/rtos.h:41-55

두 큐를 정적 메모리로 생성

task_stepmotor.c

```
step_motor_msg_q = osMessageQueueNew(
    _HW_DEF_RTOS_MSG_Q_STEP_MOTOR,
    sizeof(rtos_step_motor_msg_t),
    rtosGetStepMotorMsgQAttr()
);

step_motor_ack_q = osMessageQueueNew(
    _HW_DEF_RTOS_MSG_Q_STEP_MOTOR_ACK,
    sizeof(rtos_step_motor_ack_t),
    rtosGetStepMotorAckQAttr()
);
```

소스: stm32g474/src/ap/task/task_stepmotor.c:76-93

rtos.c - 메시지 큐 저장 공간

```
static StaticQueue_t stepMotorMsgQ_cb;
static uint8_t stepMotorMsgQ_buf[
    _HW_DEF_RTOS_MSG_Q_STEP_MOTOR * sizeof(rtos_step_motor_msg_t)
];
```

소스: stm32g474/src/bsp/rtos.c:57-79

명령과 결과의 흐름

단계	실행 태스크	핵심 코드	의미
1	Sequence	taskStepMotorMoveToFull()	메시지 작성
2	Sequence	osMessageQueuePut(msg_q)	명령을 값으로 복사
3	StepMotor	osMessageQueueGet(msg_q)	명령 수신
4	StepMotor	dm542MoveStepAsync()	짧은 PWM chunk 시작
5	ISR	APP_EVT_STEP_MOTOR_DONE	chunk 완료 통지
6	StepMotor	osMessageQueuePut(ack_q)	DONE/STOPPED/ERROR 응답
7	Sequence	taskStepMotorGetAck()	cmd_id가 같은 응답만 처리

왜 cmd_id가 필요한가

새 명령이 이전 명령을 중단할 수 있으므로 ACK가 어떤 명령의 결과인지 구분해야 합니다. cmd_id가 다른 Sequence 태스크는 그 ACK를 무시합니다.

7. Mutex: 여러 태스크가 공유하는 드라이버 보호

button.c - 재귀 Mutex와 우선순위 상속

```
static osMutexId_t button_mutex = NULL;

static const osMutexAttr_t button_mutex_attr =
{
    .name      = "button",
    .attr_bits = osMutexRecursive | osMutexPrioInherit,
};

button_mutex = osMutexNew(&button_mutex_attr);
```

소스: stm32g474/src/hw/driver/button.c:33-61

button.c - 공유 자원 접근

```
bool buttonGetPressed(uint8_t ch)
{
    bool ret = false;

    if (buttonLock() != true) return false;
    if (ch < BUTTON_MAX_CH)
    {
        ret = buttonIsPressedRaw(ch);
    }
    buttonUnlock();

    return ret;
}
```

소스: stm32g474/src/hw/driver/button.c:84-98

옵션	필요한 이유	현재 코드 사례
osMutexRecursive	같은 태스크가 동일 Mutex를 중첩 획득 가능	buttonReadData 내부에서 buttonGetPressed를 다시 호출
osMutexPrioInherit	낮은 우선순위 보유자의 우선순위를 일시 상승	우선순위 역전으로 높은 태스크가 오래 막히는 현상 완화

ISR에서는 Mutex를 사용하지 않음

buttonLock()의 ISR 검사

```
if (__get_IPSR() != 0U)
{
    return false;
}

if (osKernelGetState() == osKernelRunning)
{
    return osMutexAcquire(button_mutex, timeout) == osOK;
}
```

ISR은 잠들거나 기다릴 수 없으므로 일반 Mutex API를 호출하면 안 됩니다. DM542와 PWM에는 이를 위해 `..FromISR()` 전용 경로가 따로 있습니다.

현재 코드에 Semaphore 객체는 없음

과거 커밋 제목에는 Mutex/Semaphore가 포함되어 있지만 현재 사용처를 보면 osSemaphoreNew()은 없습니다. 실제 동기화 수단은 Mutex, EventFlags, Message Queue입니다.

8. 인터럽트와 태스크를 연결하는 코드

SN04 센서 ISR은 모터를 먼저 멈추고 이벤트를 Set

task_sensor.c - 센서 콜백

```
static void sensorSn04IsrHandler(uint8_t ch, bool detected)
{
    uint32_t evt_bit = sensorGetSn04EventBit(ch);

    if (detected == true)
    {
        if ((sensor_dm542_stop_ignore_evt & evt_bit) == 0U)
        {
            dm542StopBySensorFromISR(_DEF_DM542_1);
        }

        appEventSet(evt_bit);
    }
}
```

소스: stm32g474/src/ap/task/task_sensor.c:90-113

ISR에서는 안전을 위해 PWM을 즉시 정지합니다. 하지만 복잡한 상태 판단과 ACK 정리는 ISR이 아니라 StepMotor 태스크에서 수행합니다. ISR은 짧게 끝내고 무거운 작업을 태스크로 넘기는 전형적인 구조입니다.

ISR에서 EventFlags를 Set할 때의 내부 동작

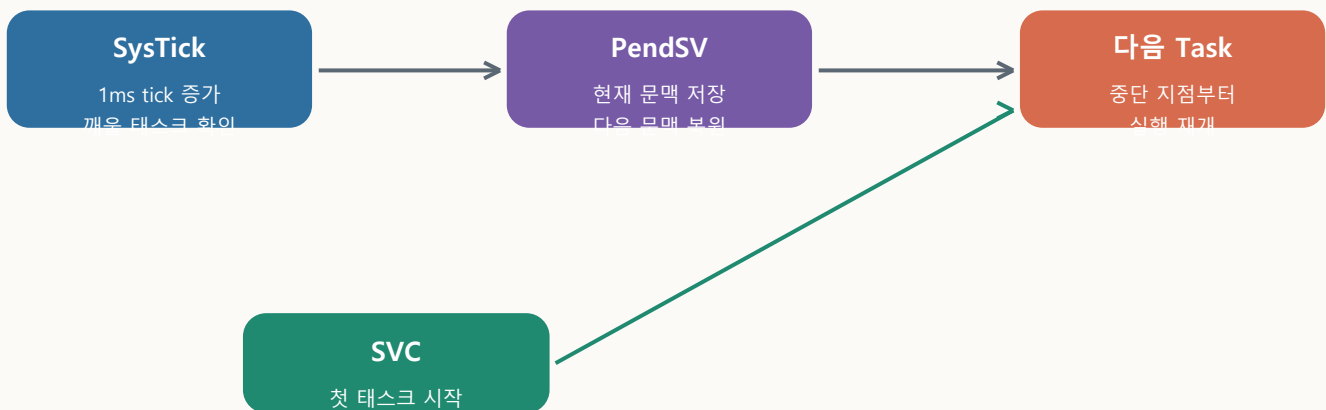
cmsis_os2.c

```
if (IS_IRQ())
{
    yield = pdFALSE;
    xEventGroupSetBitsFromISR(hEventGroup, flags, &yield);
    portYIELD_FROM_ISR(yield);
}
else
{
    xEventGroupSetBits(hEventGroup, flags);
}
```

소스: stm32g474/src/bsp/FreeRTOS/Source/CMSIS_RTOS_V2/cmsis_os2.c:1150-1178

IRQ	현재 우선순위	RTOS API 사용 가능 조건
SN04 EXTI4 / EXTI15_10	5	configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY=5 경계에서 ISR-safe API 사용
TIM2 / TIM3 PWM	5	완료 callback에서 EventFlags Set 가능
SysTick / PendSV	15	커널 전용 최저 우선순위
TIM7 HAL tick	15	HAL_IncTick만 수행

9. SysTick, PendSV, SVC가 만드는 문맥 전환



SysTick: 1ms마다 시간과 깨울 태스크 확인

port.c - xPortSysTickHandler

```

if (xTaskIncrementTick() != pdFALSE)
{
    portNVIC_INT_CTRL_REG = portNVIC_PENDSVSET_BIT;
}
  
```

소스: stm32g474/src/bsp/FreeRTOS/Source/portable/GCC/ARM_CM4F/port.c:488-505

xTaskIncrementTick()은 delay가 끝난 태스크를 Ready로 이동합니다. 더 적합한 태스크가 생기면 PendSV 인터럽트를 예약합니다.

PendSV: 현재 태스크 저장 후 다음 태스크 복원

port.c - 핵심 assembly 흐름

```

mrs    r0, psp
stmdb  r0!, {r4-r11, r14}    // save current task context
str    r0, [r2]              // save stack pointer in TCB

bl     vTaskSwitchContext    // select next Ready task

ldmia  r0!, {r4-r11, r14}    // restore next task context
msr    psp, r0
  
```

소스: stm32g474/src/bsp/FreeRTOS/Source/portable/GCC/ARM_CM4F/port.c:431-485

SVC: 스케줄러 시작 시 첫 태스크 실행

port.c - 첫 태스크 시작

```

svc 0

// vPortSVCHandler restores the first task stack
ldmia r0!, {r4-r11, r14}
msr   psp, r0
  
```

문맥 전환의 핵심

태스크가 함수 처음부터 다시 실행되는 것이 아닙니다. 각 태스크의 스택에 CPU 레지스터와 복귀 위치를 저장하므로, 다시 선택되면 이전에 중단된 코드 다음 줄부터 계속 실행됩니다.

10. 커널이 자동 생성하는 내부 태스크

Idle Task

tasks.c - scheduler 시작 시 자동 생성

```
xIdleTaskHandle = xTaskCreateStatic(
    prvIdleTask,
    configIDLE_TASK_NAME,
    ulIdleTaskStackSize,
    NULL,
    tskIDLE_PRIORITY,
    pxIdleTaskStackBuffer,
    pxIdleTaskTCBBuffer
);
```

소스: stm32g474/src/bsp/FreeRTOS/Source/tasks.c:1975-2016

Idle Task 역할	현재 프로젝트에서의 의미
실행할 Ready 태스크가 없을 때 실행	Button/Sensor/Step/main/LED/Sequence가 모두 Blocked일 때 CPU 사용
삭제된 태스크 메모리 정리	향후 vTaskDelete/osThreadTerminate 사용 시 정리 담당
Idle Hook 또는 저전력 진입	현재 configUSE_IDLE_HOOK=0, tickless idle도 비활성

Timer Service Task

FreeRTOSConfig.h

```
#define configUSE_TIMERS          1
#define configTIMER_TASK_PRIORITY 2
#define configTIMER_QUEUE_LENGTH 10
#define configTIMER_TASK_STACK_DEPTH 256
```

현재 애플리케이션은 osTimerNew()을 직접 만들지 않습니다. 하지만 ISR에서 EventFlags를 Set할 때 FreeRTOS가 Timer Service Task에 지연 처리 요청을 전달할 수 있어 간접적으로 사용됩니다.

Idle/Timer Task의 정적 메모리

cmsis_os2.c - wrapper가 제공하는 weak 기본 구현

```
static StaticTask_t Idle_TCB;
static StackType_t Idle_Stack[configMINIMAL_STACK_SIZE];

static StaticTask_t Timer_TCB;
static StackType_t Timer_Stack[configTIMER_TASK_STACK_DEPTH];
```

소스: stm32g474/src/bsp/FreeRTOS/Source/CMSIS_RTOS_V2/cmsis_os2.c:2450-2481

11. 내가 직접 추가한 코드 지도

파일	추가한 코드	FreeRTOS 의미
src/main.c	osKernelInitialize, mainThread 생성, osKernelStart	커널 시작점
src/common/rtos_def.h	태스크 우선순위, 스택, 큐 길이	RTOS 자원 정책
src/bsp/rtos.c	StaticTask_t, StackType_t, StaticQueue_t	정적 메모리 제공
src/ap/task/task_manager.c	각 Init 함수 순차 호출	애플리케이션 RTOS 객체 생성 순서
src/ap/task/task_button.c	10ms polling, EventFlags Set	주기 태스크와 이벤트 생산자
src/ap/task/task_sensor.c	10ms 상태 동기화, ISR callback	Polling + ISR 혼합
src/ap/task/task_stepmotor.c	명령/ACK 큐, 비동기 chunk	작업 태스크와 메시지 통신
src/ap/task/task_sequence.c	Sequence thread 생성	상태 머신 전용 태스크
src/ap/app_sequence.c	Event wait, ACK wait, 상태 전이	이벤트 소비자와 장비 오케스트레이션
각 HW driver	Recursive Mutex, FromISR 경로	공유 자원 보호와 ISR 분리

추천 코드 읽기 순서

순서	파일	확인할 질문
1	main.c	커널을 언제 초기화하고 시작하는가?
2	rtos_def.h / rtos.c	각 태스크의 우선순위와 스택은 얼마인가?
3	task_manager.c	어떤 순서로 Event/Thread/Queue를 만드는가?
4	task_button.c	주기 태스크가 어떻게 delay로 CPU를 반납하는가?
5	app_event.c / app_sequence.c	이벤트 생산자와 소비자는 누구인가?
6	task_stepmotor.c	Queue, ACK, ISR 완료 이벤트가 어떻게 연결되는가?
7	driver lock 함수	Mutex와 FromISR 경로를 왜 분리했는가?
8	cmsis_os2.c / port.c	CMSIS API가 실제 커널과 CPU 문맥 전환으로 어떻게 이어지는가?

12. 현재 코드에서 기억할 주의점

1. rtosInit()은 현재 커널 초기화 함수가 아님

src/bsp/rtos.c의 rtosInit()은 true만 반환하고 호출되지 않습니다. 실제 초기화는 osKernelInitialize()입니다.

2. CubeMX defaultTask는 현재 비활성

CubeMX 생성 원본과 현재 직접 작성 코드가 함께 남아 있습니다. 코드를 추적할 때 src/lib/cube_g474/Core/Src/app_freertos.c를 실제 실행 코드로 착각하지 않아야 합니다.

3. Static + Dynamic 혼합 구성

애플리케이션 Thread, Queue, EventFlags는 정적 메모리입니다. 하지만 driver Mutex는 cb_mem을 제공하지 않아 heap_4에서 동적으로 생성됩니다.

4. 동일 Normal 우선순위가 많음

main, Button, Sensor, StepMotor, Sequence가 모두 Normal입니다. 대부분 delay/wait로 잘 Block되지만, 무한 루프 안에서 대기 없이 오래 실행되는 코드가 추가되면 같은 우선순위 태스크의 반응성이 떨어질 수 있습니다.

5. 빌드 구조에 FreeRTOS 복사본이 둘 있음

커널 소스는 src/bsp/FreeRTOS/Source를 빌드하지만 헤더 include 경로는 src/lib/cube_g474/Middlewares/Third_Party/FreeRTOS를 가리킵니다. 두 복사본을 서로 다른 버전으로 수정하면 ABI나 설정 불일치가 생길 수 있습니다.

한 줄 정리: 이 프로젝트에서 FreeRTOS 적용의 본질은 기능별 무한 루프를 태스크로 분리하고, 태스크가 기다릴 때 delay/event/queue로 CPU를 반납하며, 공유 드라이버는 Mutex로 보호하고, ISR은 짧게 처리한 뒤 이벤트로 태스크를 깨우는 구조입니다.

End of guide